

SRE CASE STUDY

AZURE ARC HYBRID CLOUD LAB

Projekt Proof-of-Value demonstrujący hybrydową orkiestrację chmury, observability, FinOps i praktyki Infrastructure-as-Code przy użyciu K3s Kubernetes i Microsoft Azure Arc.

Autor: Bartosz Suszko

Firma: IT Solutions Bartosz Suszko, Olsztyn

Data: Maj 2026

GitHub: github.com/buth11/sre-homelab-azure-arc

Stos: K3s v1.35.4 | Azure Arc | Prometheus | Grafana | Terraform | KQL

Spis treści

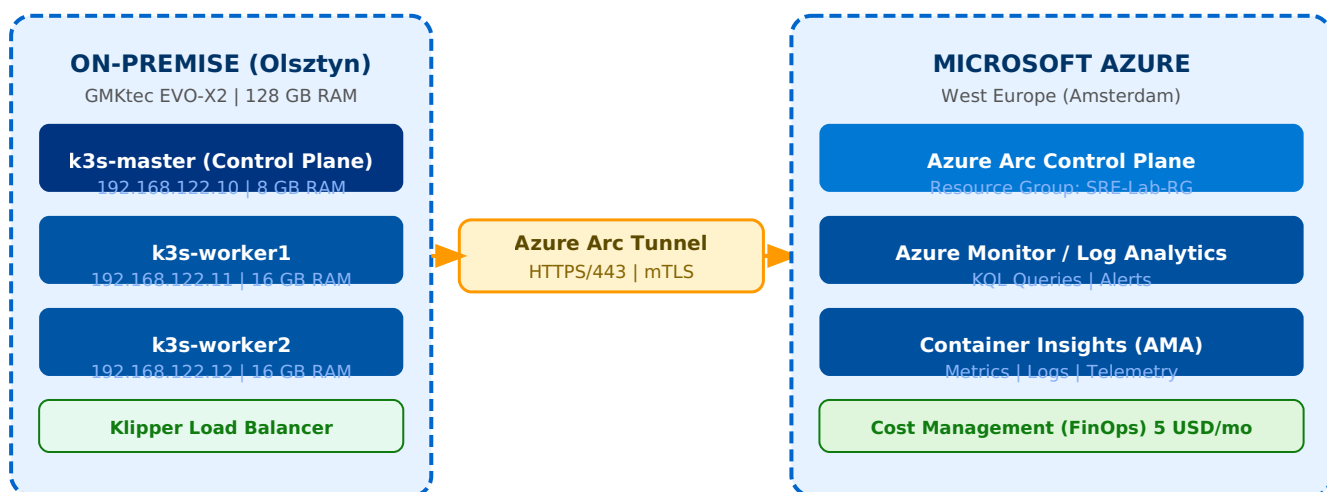
1. Kontekst biznesowy i architektura hybrydowa
2. Infrastruktura On-Premise — klaster K3s
3. FinOps — kontrola kosztów w chmurze
4. Integracja hybrydowa — Azure Arc
5. Observability — Container Insights i Azure Monitor
6. Zapytania KQL — diagnostyka i SLO
7. Load Balancing — weryfikacja round-robin
8. Prometheus i Grafana — lokalny monitoring stack
9. Infrastructure as Code — Terraform
10. GitHub — dokumentacja i automatyzacja
11. Postmortem — POST-001 etcd Split-Brain
12. Wyniki i wnioski

Rozdział 1: Kontekst biznesowy i architektura hybrydowa

Projekt demonstruje budowę i operowanie produkcyjnym środowiskiem hybrydowym łączącym infrastrukturę on-premise z chmurą Microsoft Azure. Środowisko zostało zbudowane na dedykowanym sprzęcie (GMKtec EVO-X2, 128 GB RAM) w oparciu o technologie używane w komercyjnych wdrożeniach enterprise SaaS.

Celem projektu jest udowodnienie kompetencji SRE w obszarach: zarządzania hybrydową infrastrukturą, observability, automatyzacji i kontroli kosztów — wszystkich wymaganych przy utrzymaniu platformy SaaS dla klientów enterprise.

Diagram architektury



Stos technologiczny

Warstwa	Technologia	Zastosowanie
Hypervisor	KVM na Fedora 43	Wirtualizacja węzłów klastra
System operacyjny	Ubuntu Server 24.04 LTS	Wszystkie węzły K3s
Kubernetes	K3s v1.35.4+k3s1	Lekka dystrybucja K8s
Cloud Bridge	Azure Arc (agent v1.33.0)	Hybrydowa płaszczyzna kontrolna
Monitoring cloud	Azure Monitor + Container Insights	Telemetry i KQL
Monitoring lokalny	Prometheus + Grafana v3.21	Open-source observability stack
IaC	Terraform ~3.100	Infrastructure as Code
Package manager	Helm v3.21.0	Wdrażanie aplikacji K8s
Load Balancer	K3s Klipper LB	Round-robin L4
FinOps	Azure Cost Management	Budżet 5 USD/miesiąc

Rozdział 2: Infrastruktura On-Premise — klaster K3s

Klaster K3s składa się z trzech maszyn wirtualnych uruchomionych na GMKtec EVO-X2 z 128 GB RAM. Każda maszyna ma statyczny adres IP w podsieci 192.168.122.0/24 i zainstalowany system Ubuntu Server 24.04 LTS.

Węzeł	Rola	IP	CPU	RAM	Dysk
k3s-master	Control Plane	192.168.122.10	4 vCPU	8 GB	40 GB
k3s-worker1	Worker	192.168.122.11	4 vCPU	16 GB	60 GB
k3s-worker2	Worker	192.168.122.12	4 vCPU	16 GB	60 GB

Instalacja klastra została zautomatyzowana skryptami Bash zawartymi w repozytorium. Skrypt instalacyjny mastera wykonuje preflight check nazwy hosta (zabezpieczenie przed błędami etcd), konfiguruje parametry sysctl, instaluje K3s i automatycznie generuje token dla węzłów roboczych. Skrypt workera waliduje format nazwy hosta przez wyrażenie regularne (k3s-workerN) i sprawdza osiągalność API mastera przed dołączeniem do klastra.

Rozdział 3: FinOps — kontrola kosztów w chmurze

Zgodnie z najlepszymi praktykami SRE, konfiguracja budżetów i alertów kosztowych została wykonana jako pierwsza czynność — przed uruchomieniem jakichkolwiek zasobów w chmurze. Zapobiega to niekontrolowanym wydatkom, co jest szczególnie istotne w środowiskach SaaS zarządzających infrastrukturą wielu klientów jednocześnie.

The screenshot shows the Microsoft Azure Cost Management console for user Bartosz Suszko. The 'Budżety' (Budgets) section is active, displaying a table with one budget entry:

Nazwa	Zakres	Resetuj okres	Data utworzenia	Data wygaśnięcia	Budżet	Prognostyczne	Oszacowane wy...	Postęp
SRE-Lab-Budget	55df28b8-39ed-58b...	Monthly	1.05.2026	30.04.2028	5.00 USD	0	\$0.00	0.00%

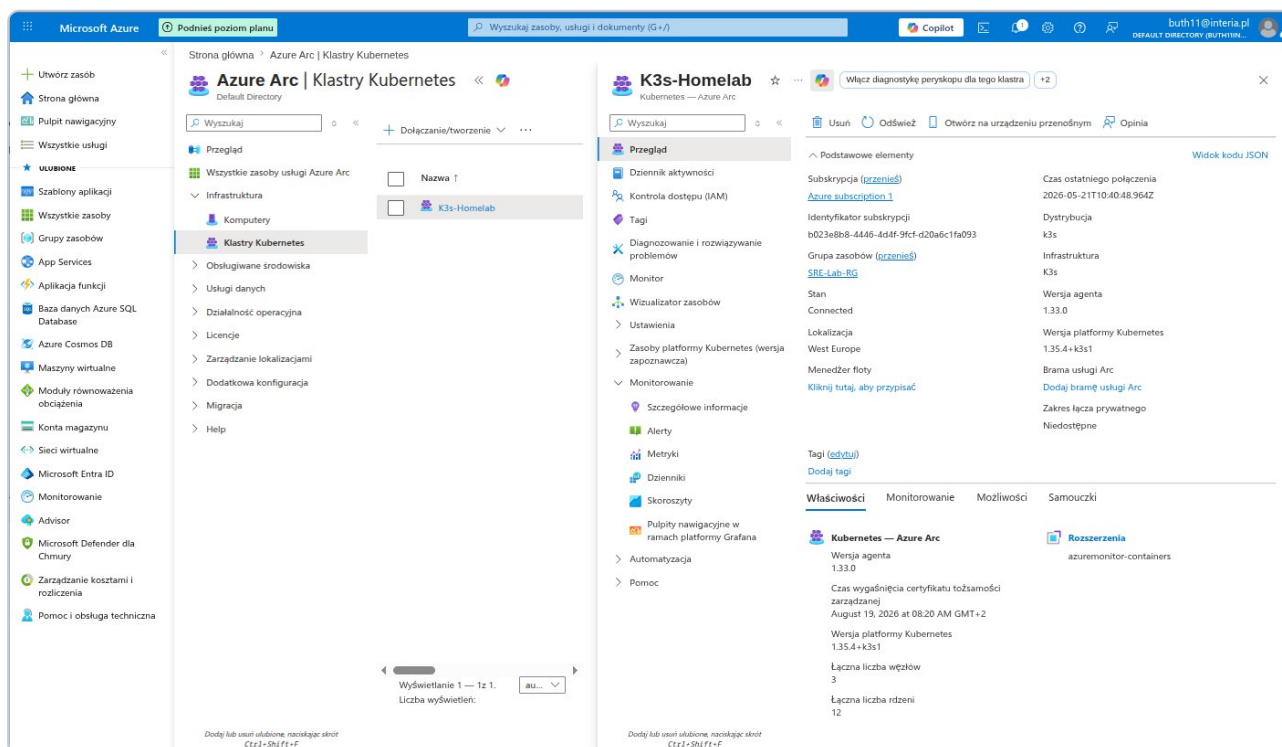
Below the table, there are sections for 'Alerty dotyczące kosztów' (Cost alerts) and 'Reguły alertów' (Alert rules), with a 'Budżety' (Budgets) link highlighted in the left sidebar.

Zrzut 1: Azure Cost Management — budżet SRE-Lab-Budget (5 USD/mies.) z alertami e-mail przy 50% i 100% progu. Stan końcowy: 0.00 USD przez cały okres testów.

Skonfigurowane progi alertów: powiadomienie e-mail przy osiągnięciu 50% budżetu (2.50 USD) oraz 100% (5.00 USD). Środowisko nie wygenerowało żadnych kosztów przez cały okres prowadzenia badań.

Rozdział 4: Integracja hybrydowa — Azure Arc

Azure Arc umożliwia zarządzanie lokalnym klastrem Kubernetes z poziomu portalu Azure. Agent Arc zainstalowany na klastrze K3s nawiązuje połączenie wychodzące (HTTPS/443, mTLS) do infrastruktury Azure, co eliminuje konieczność otwierania portów przychodzących na firewallu.

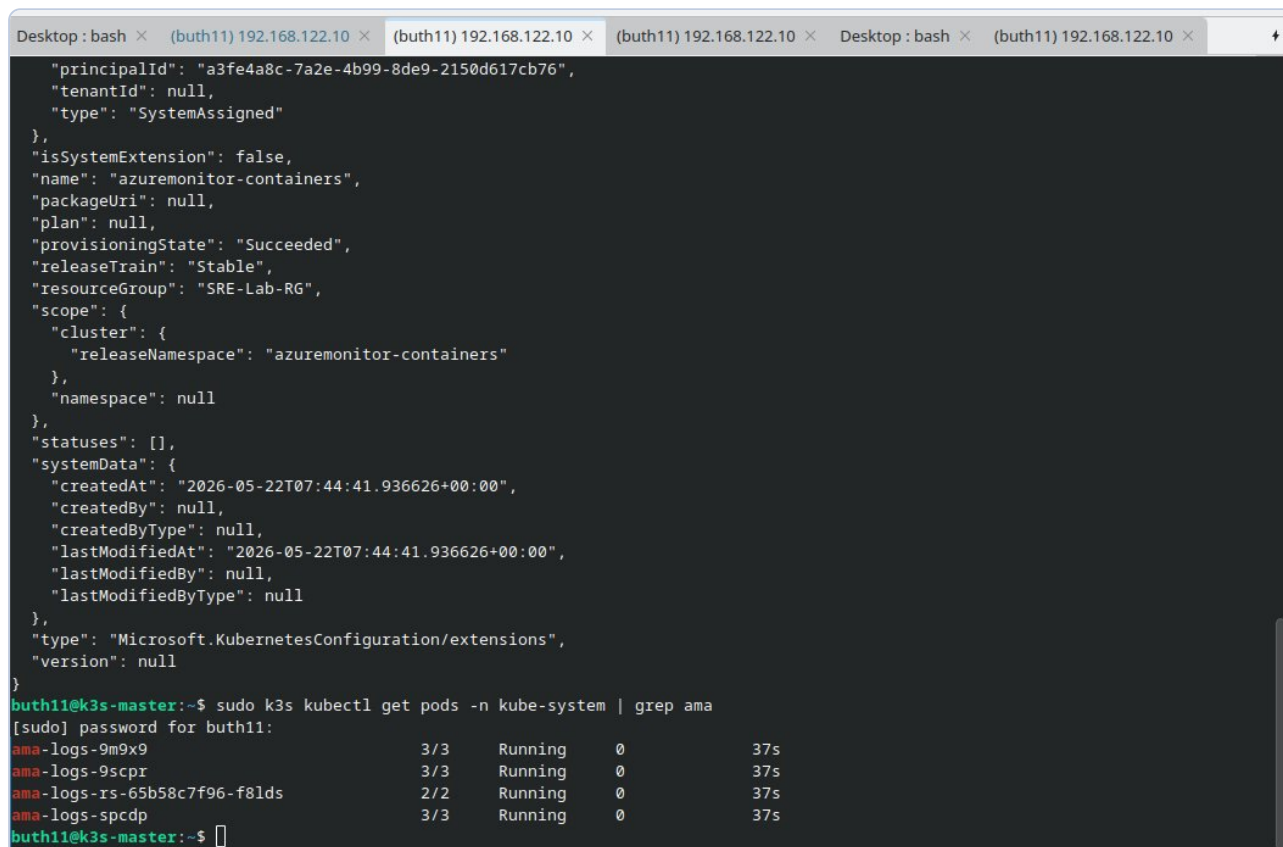


Zrzut 2: Azure Arc — klastre K3s-Homelab ze statusem Connected. Dystrybucja: k3s v1.35.4+k3s1, 3 węzły, 12 rdzeni, rozszerzenie azuremonitor-containers aktywne.

Po restarcie maszyn wirtualnych klastre automatycznie reconnectował się z Azure Arc w ciągu około 3 minut bez żadnej interwencji manualnej — agenty Arc działają jako systemowe serwisy uruchamiane przy starcie systemu.

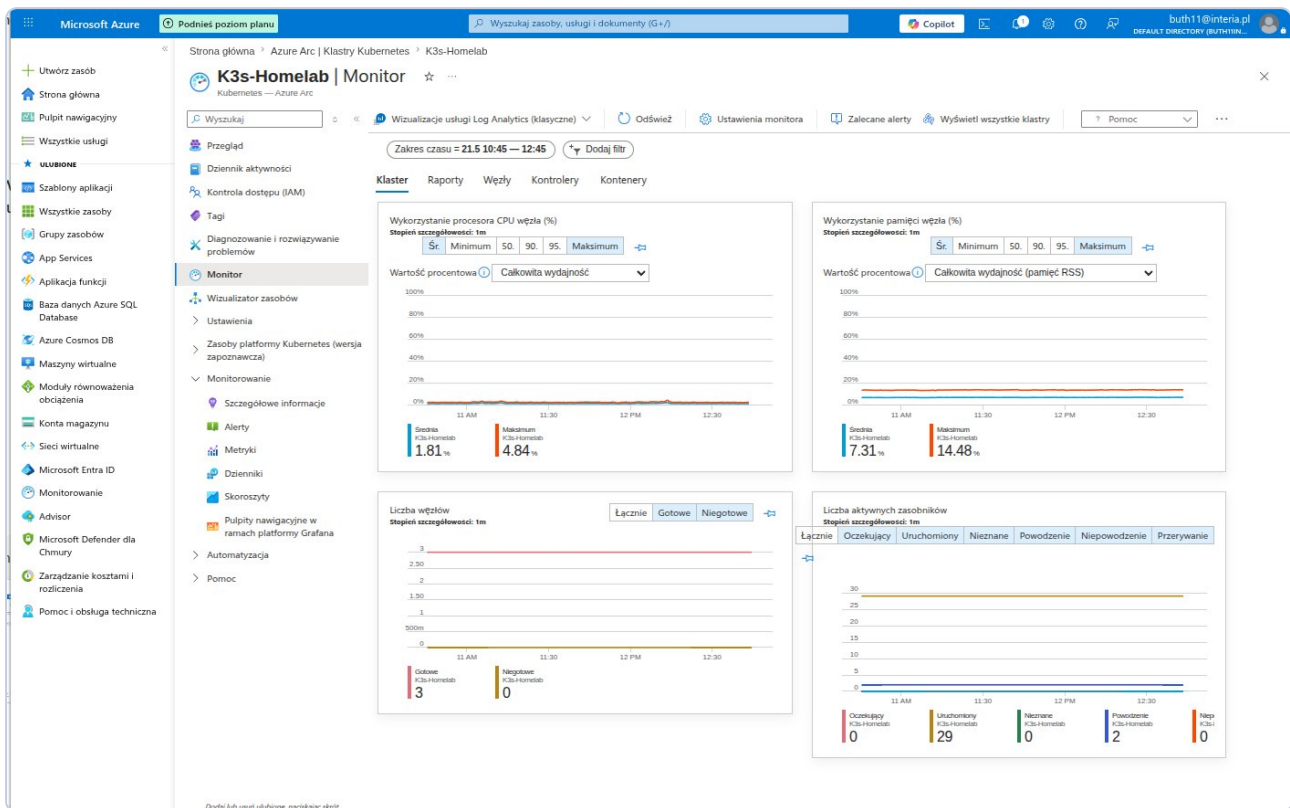
Rozdział 5: Observability — Container Insights i Azure Monitor

Container Insights został wdrożony jako rozszerzenie Azure Arc za pomocą jednego polecenia CLI. Na klastrze uruchomiły się 4 pody agentów AMA (Azure Monitor Agent): trzy instancje DaemonSet (po jednym na każdy węzeł) oraz jeden ReplicaSet dla metryk zbiorczych.



```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop: bash × (buth11) 192.168.122.10 ×
{"principalId": "a3fe4a8c-7a2e-4b99-8de9-2150d617cb76",
 "tenantId": null,
 "type": "SystemAssigned"
},
"isSystemExtension": false,
"name": "azuremonitor-containers",
"packageUri": null,
"plan": null,
"provisioningState": "Succeeded",
"releaseTrain": "Stable",
"resourceGroup": "SRE-Lab-RG",
"scope": {
  "cluster": {
    "releaseNamespace": "azuremonitor-containers"
  },
  "namespace": null
},
"statuses": [],
"systemData": {
  "createdAt": "2026-05-22T07:44:41.936626+00:00",
  "createdBy": null,
  "createdByType": null,
  "lastModifiedAt": "2026-05-22T07:44:41.936626+00:00",
  "lastModifiedBy": null,
  "lastModifiedByType": null
},
"type": "Microsoft.KubernetesConfiguration/extensions",
"version": null
}
buth11@k3s-master:~$ sudo k3s kubectl get pods -n kube-system | grep ama
[sudo] password for buth11:
ama-logs-9m9x9          3/3      Running    0          37s
ama-logs-9scpr          3/3      Running    0          37s
ama-logs-rs-65b58c7f96-f81ds 2/2      Running    0          37s
ama-logs-spcdp          3/3      Running    0          37s
buth11@k3s-master:~$
```

Zrzut 3: Pody AMA (ama-logs) w przestrzeni kube-system — 4 agenty Running, 0 restartów, 37 sekund po instalacji.



Zrzut 4: Azure Monitor Container Insights — CPU avg 1.58%/max 2.30%, RAM avg 6.27%/max 12.01%, węzły Gotowe: 3/3, aktywne zasobniki: 29.

Rozdział 6: Zapytania KQL — diagnostyka i SLO

KQL (Kusto Query Language) jest podstawowym narzędziem analitycznym SRE w ekosystemie Azure Monitor. Poniżej przedstawiono 5 zapytań diagnostycznych wykonanych na rzeczywistych danych klastra, demonstrując podejście do monitoringu oparte na danych.

Zapytanie 1 — Inwentarz węzłów klastra

```
KubeNodeInventory
| where TimeGenerated > ago(1h)
| summarize arg_max(TimeGenerated, *) by Computer
| project Computer, Status, KubeletVersion
| order by Computer asc
```

The screenshot displays the Microsoft Azure portal interface. On the left, the 'Log Analytics' workspace is selected. The main pane shows a KQL query editor with the following query:

```
1 KubeNodeInventory
2 | where TimeGenerated > ago(1h)
3 | summarize arg_max(TimeGenerated, *) by Computer
4 | project
5 |   Computer,
6 |   Status,
7 |   KubeletVersion
8 | order by Computer asc
```

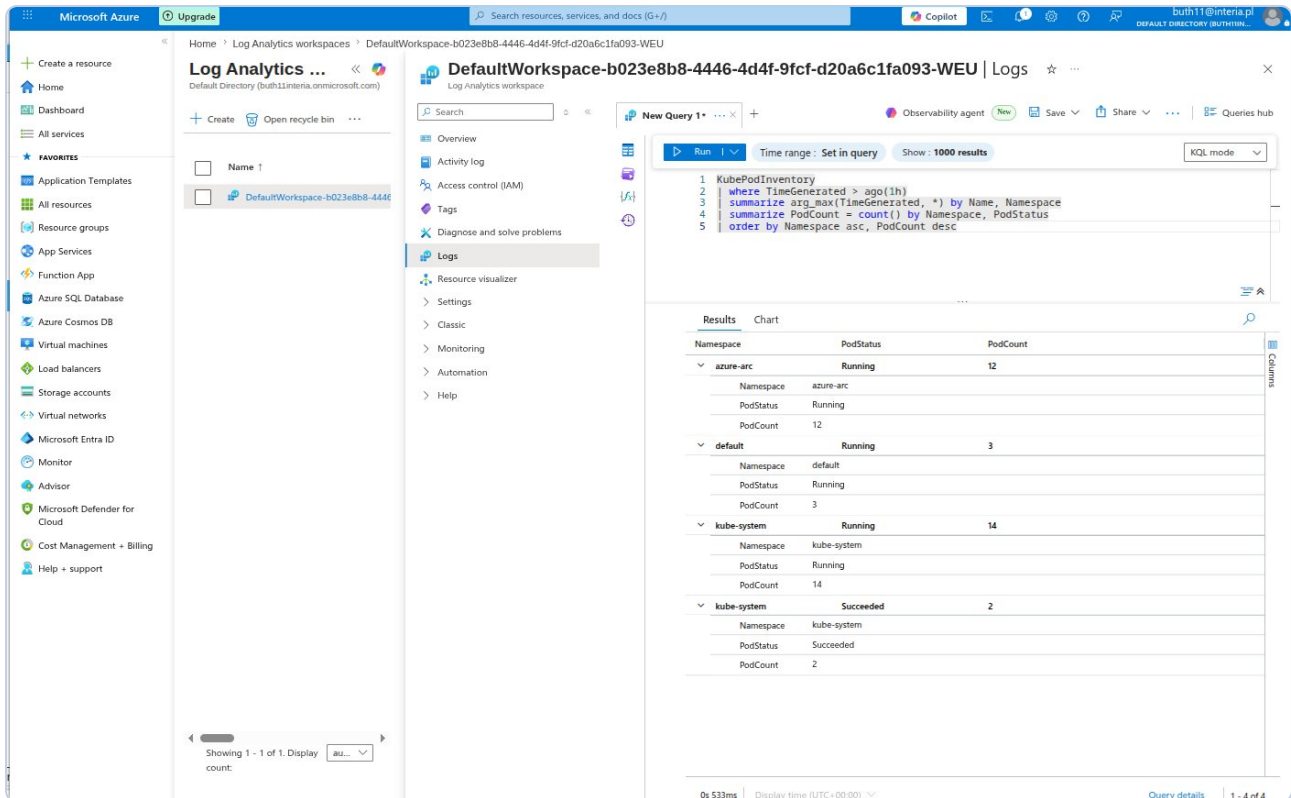
The query results are displayed in a table with columns: Computer, Status, and KubeletVersion. The results show three nodes:

Computer	Status	KubeletVersion
k3s-master	Ready	v1.35.4+k3s1
k3s-worker1	Ready	v1.35.4+k3s1
k3s-worker2	Ready	v1.35.4+k3s1

KQL-1: Inwentarz węzłów — k3s-master, k3s-worker1, k3s-worker2 — wszystkie STATUS=Ready, wersja v1.35.4+k3s1.

Zapytanie 2 — Status podów per namespace

```
KubePodInventory
| where TimeGenerated > ago(1h)
| summarize arg_max(TimeGenerated, *) by Name, Namespace
| summarize PodCount = count() by Namespace, PodStatus
| order by Namespace asc, PodCount desc
```



The screenshot shows the Microsoft Azure portal interface. On the left is the navigation pane with 'Log Analytics' selected. The main area displays a 'New Query' window for a Log Analytics workspace. The query is as follows:

```
1 KubePodInventory
2 | where TimeGenerated > ago(1h)
3 | summarize arg_max(TimeGenerated, *) by Name, Namespace
4 | summarize PodCount = count() by Namespace, PodStatus
5 | order by Namespace asc, PodCount desc
```

The results are displayed in a table with columns: Namespace, PodStatus, and PodCount. The results are grouped by Namespace.

Namespace	PodStatus	PodCount
azure-arc	Running	12
default	Running	3
kube-system	Running	14
kube-system	Succeeded	2

KQL-2: azure-arc: 12 Running | default: 3 Running (aplikacja demo) | kube-system: 14 Running + 2 Succeeded.

Zapytanie 3 — Historia eventów (24h)

```
KubeEvents
| where TimeGenerated > ago(24h)
| project TimeGenerated, Namespace, Name, Reason, Message
| order by TimeGenerated desc
| take 20
```

The screenshot shows the Microsoft Azure portal interface. On the left is the navigation pane with options like 'Create a resource', 'Home', 'Dashboard', 'All services', and 'FAVORITES'. The main area displays the 'Log Analytics workspace' for 'DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU'. A query named 'KubeEvents' is executed, showing results for the last 24 hours. The results table has columns: TimeGenerated [UTC], Namespace, Name, and Reason. The events listed include 'FailedMount', 'Unhealthy', 'Rebooted', and 'FailedScheduling'.

TimeGenerated [UTC]	Namespace	Name	Reason
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-9scpr	FailedMount
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-rs-65b58c7f96-f8bds	FailedMount
5/22/2026, 7:35:32.000 AM	azure-arc	kube-aad-proxy-5f59465457-i2...	Unhealthy
5/22/2026, 7:35:25.000 AM	azure-arc	config-agent-75548f59c5-k4zrv	Unhealthy
5/22/2026, 7:35:13.000 AM		k3s-worker2	Rebooted
5/22/2026, 7:35:11.000 AM		k3s-worker1	Rebooted
5/22/2026, 7:35:09.000 AM	kube-system	metrics-server-786d997795-zz...	Unhealthy
5/22/2026, 7:35:07.000 AM		k3s-master	Rebooted
5/21/2026, 10:53:39.000 AM	kube-system	ama-logs-89qgj	Unhealthy
5/21/2026, 10:52:38.000 AM	kube-system	ama-logs-rs-65b58c7f96-nixhw	Unhealthy
5/21/2026, 8:37:39.000 AM	svcib-aras-demo-app-6f6f7352...		FailedScheduling
5/21/2026, 8:37:39.000 AM	svcib-aras-demo-app-6f6f7352...		FailedScheduling
5/21/2026, 8:37:39.000 AM	svcib-aras-demo-app-6f6f735...		FailedScheduling

KQL-3: 13 eventów z 24h — Rebooted (restart VM), chwilowe Unhealthy po restarcie agentów Arc, FailedScheduling Klipper LB. Brak krytycznych błędów aplikacyjnych.

Zapytanie 4 — Pody z restartami po awarii

```
KubePodInventory
| where TimeGenerated > ago(24h)
| summarize arg_max(TimeGenerated, *) by Name, Namespace
| where PodRestartCount > 0
| project Namespace, Name, PodRestartCount, PodStatus
| order by PodRestartCount desc
```

The screenshot shows the Microsoft Azure portal interface. On the left is the navigation pane with options like 'Create a resource', 'Home', 'Dashboard', and 'All services'. The main area displays the 'Log Analytics workspace' for 'DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU'. A KQL query is entered in the 'New Query' editor:

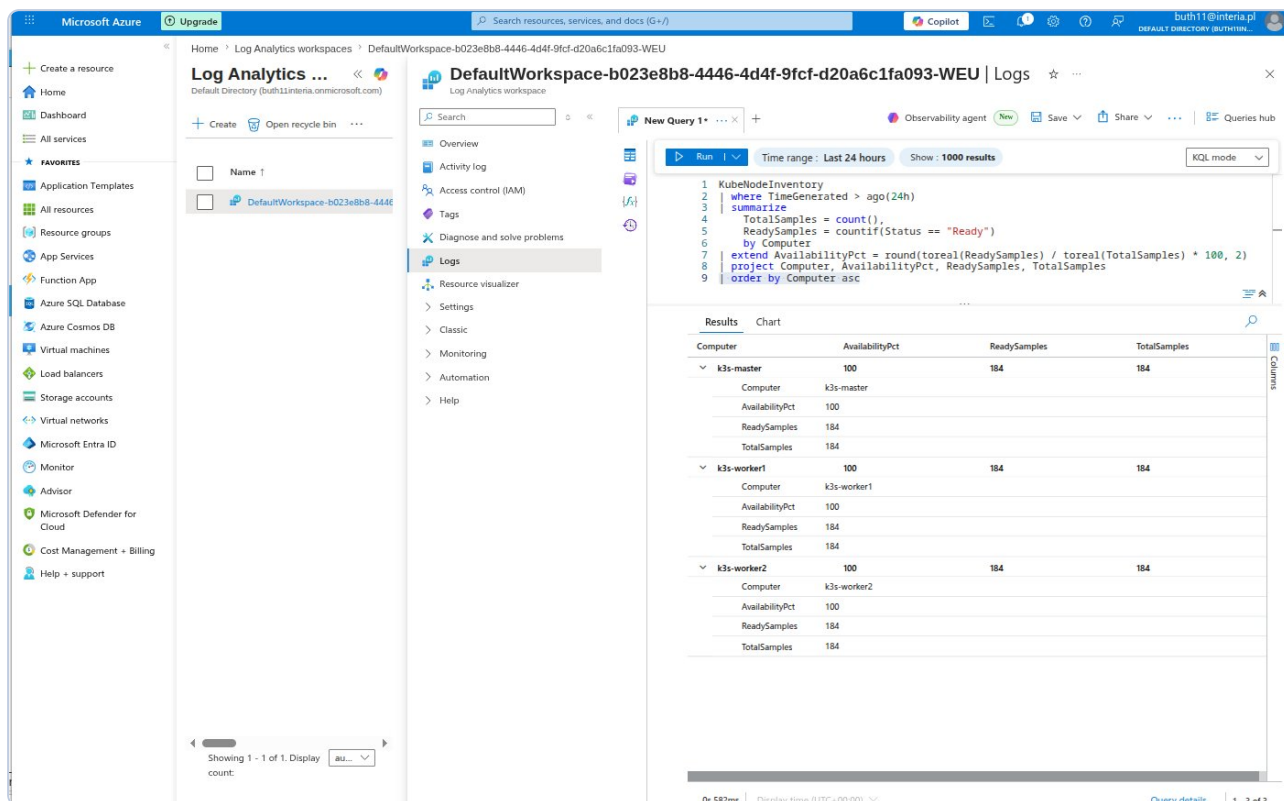
```
1 KubePodInventory
2 | where TimeGenerated > ago(24h)
3 | summarize arg_max(TimeGenerated, *) by Name, Namespace
4 | where PodRestartCount > 0
5 | project Namespace, Name, PodRestartCount, PodStatus
6 | order by PodRestartCount desc
```

The results are shown in a table with columns: Namespace, Name, PodRestartCount, and PodStatus. The table lists various pods across different namespaces, including 'azure-arc' and 'kubernetes-system', with their respective restart counts and current status (mostly 'Running').

KQL-4: 27 podów z restartami po restarcie VM — wszystkie w stanie Running. Klaster samodzielnie odtworzył pełną pojemność bez interwencji operatora.

Zapytanie 5 — Obliczenie SLO (okno 24h)

```
KubeNodeInventory
| where TimeGenerated > ago(24h)
| summarize
    TotalSamples = count(),
    ReadySamples = countif(Status == "Ready")
    by Computer
| extend AvailabilityPct = round(toreal(ReadySamples) / toreal(TotalSamples) * 100, 2)
| project Computer, AvailabilityPct, ReadySamples, TotalSamples
| order by Computer asc
```

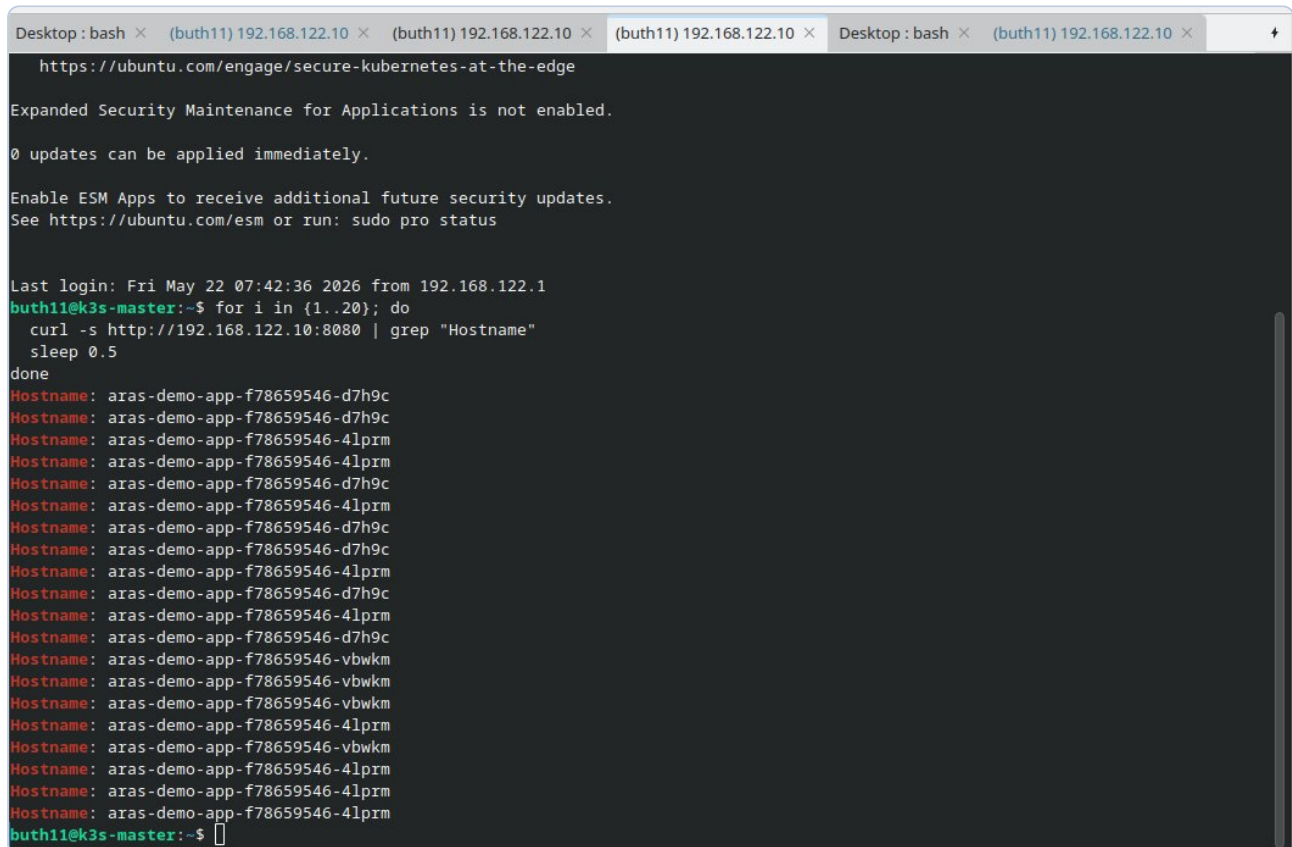


KQL-5: SLO Availability — k3s-master: 100%, k3s-worker1: 100%, k3s-worker2: 100%. Wszystkie 184/184 próbki Ready. Error budget nienaruszony.

Wynik SLO: 100% dostępności wszystkich węzłów w oknie 24h. Target dla środowisk enterprise: 99,9% (43 min error budget/miesiąc). Klaster nie wykorzystał żadnej części budżetu błędów — nawet po pełnym restarcie infrastruktury VM.

Rozdział 7: Load Balancing — weryfikacja round-robin

Klipper Load Balancer (wbudowany w K3s) rozdziela ruch na wszystkie 3 repliki aplikacji demo przez mechanizm iptables. Weryfikacja przeprowadzona przez 20 kolejnych żądań HTTP z obserwacją nazw hostów w odpowiedziach.



```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop: bash × (buth11) 192.168.122.10 ×
https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

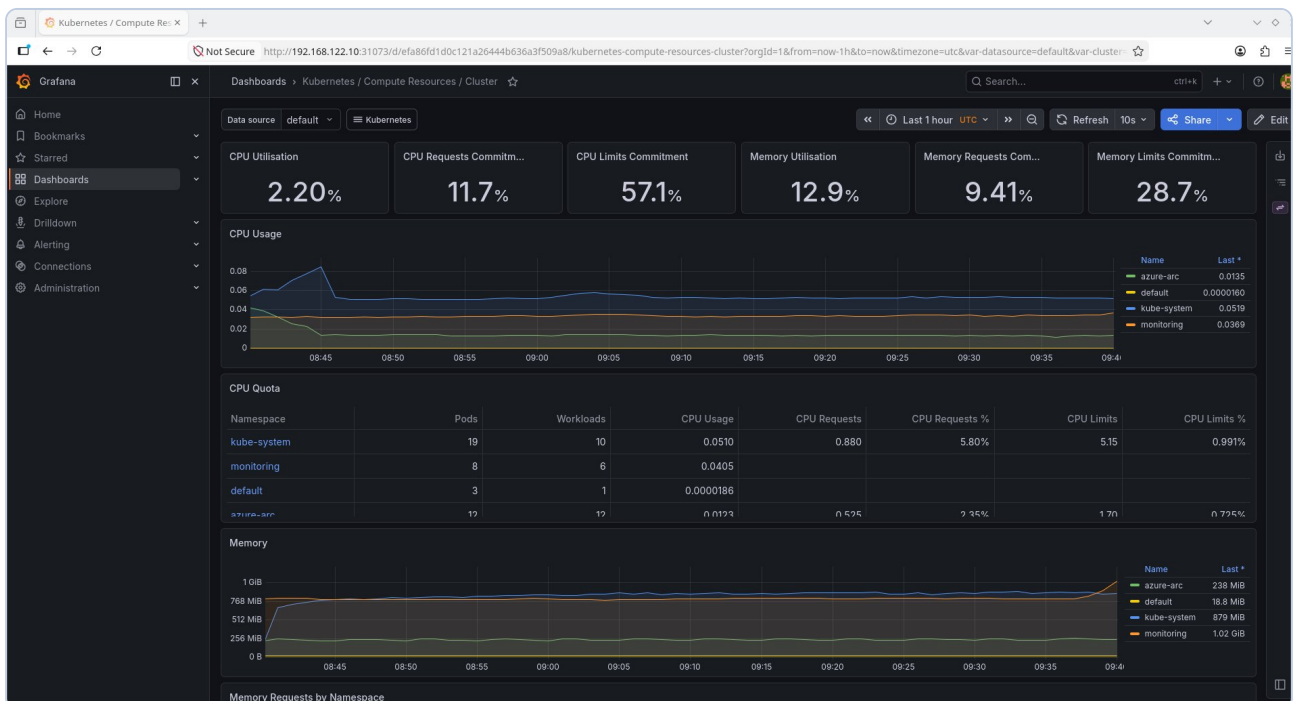
Last login: Fri May 22 07:42:36 2026 from 192.168.122.1
buth11@k3s-master:~$ for i in {1..20}; do
  curl -s http://192.168.122.10:8080 | grep "Host"
  sleep 0.5
done
Host: aras-demo-app-f78659546-d7h9c
Host: aras-demo-app-f78659546-d7h9c
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-d7h9c
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-d7h9c
Host: aras-demo-app-f78659546-d7h9c
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-d7h9c
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-d7h9c
Host: aras-demo-app-f78659546-vbwkm
Host: aras-demo-app-f78659546-vbwkm
Host: aras-demo-app-f78659546-vbwkm
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-vbwkm
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-4lprm
Host: aras-demo-app-f78659546-4lprm
buth11@k3s-master:~$
```

Zrzut 5: Round-robin LB — 20 requestów na port 8080 rozłożonych między 3 pody: d7h9c, 4lprm, vbwkm. Równoległa generacja 900 requestów (3x300) bez żadnego błędu.

Rozdział 8: Prometheus i Grafana — lokalny monitoring stack

Lokalny stos monitoringu został zainstalowany przez Helm chart `kube-prometheus-stack`. Jedna komenda wdrożyła: Prometheus, Grafanę, Alertmanager, kube-state-metrics oraz node-exporter na wszystkich węzłach.

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.adminPassword=SRE-Lab-2026 \
  --set prometheus.prometheusSpec.retention=24h
```



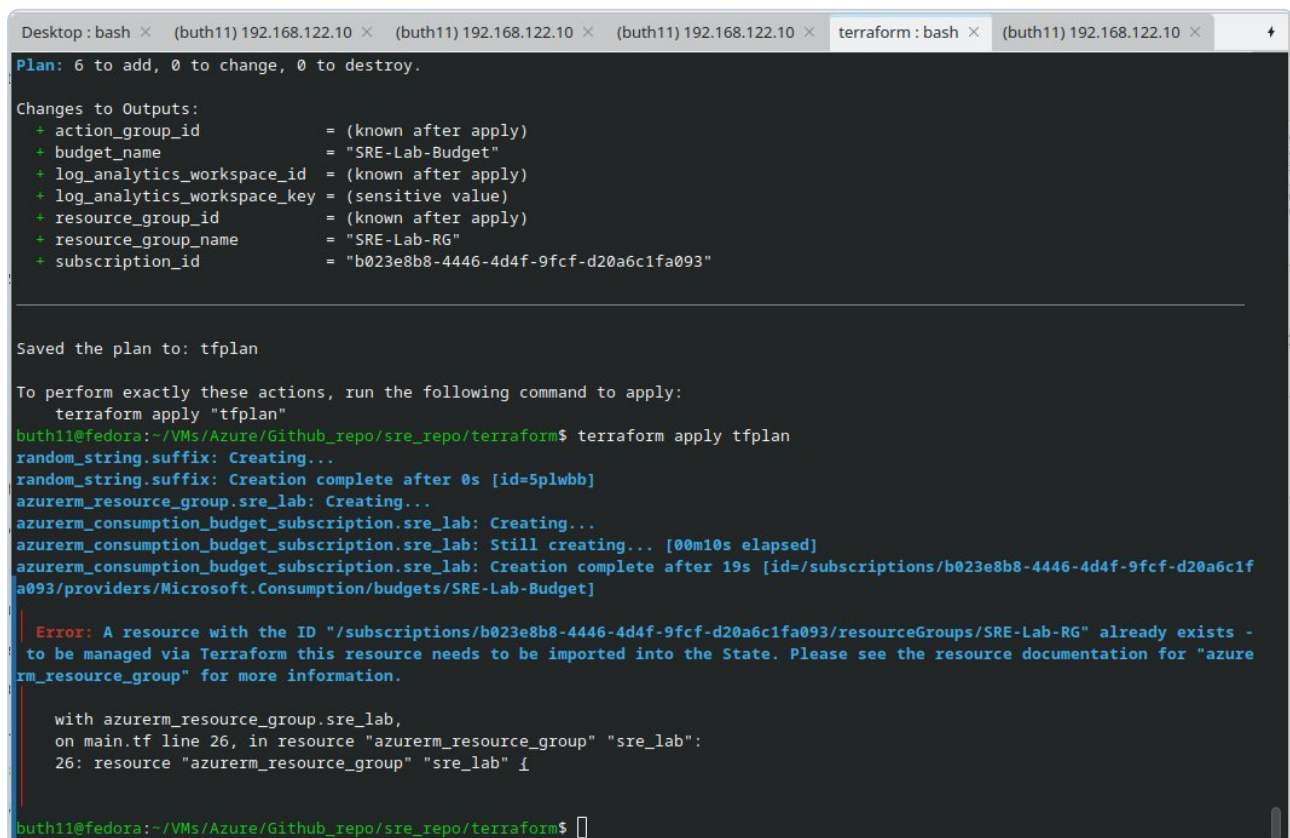
Zrzut 6: Grafana — dashboard "Kubernetes / Compute Resources / Cluster": CPU Utilisation 2.20%, Memory 12.9%, tabela CPU Quota per namespace (kube-system: 19 podów, monitoring: 8, default: 3, azure-arc: 12).

Rozdział 9: Infrastructure as Code — Terraform

Cała infrastruktura Azure została zdefiniowana w kodzie Terraform. Podejście IaC zapewnia powtarzalność wdrożeń, audit trail zmian przez Git i eliminuje błędy manualne wynikające z ręcznej pracy w portalu.

Zasoby zarządzane przez Terraform

- **azurerm_resource_group** — SRE-Lab-RG w West Europe z tagami (managed_by=terraform)
- **azurerm_log_analytics_workspace** — retencja 30 dni, SKU PerGB2018
- **azurerm_consumption_budget_subscription** — 5 USD/mies. z alertami 50% i 100%
- **azurerm_monitor_action_group** — email na buth11@interia.pl
- **azurerm_monitor_scheduled_query_rules_alert_v2** — alert OOMKill (zapytanie KQL)



```
Desktop: bash x (buth11) 192.168.122.10 x (buth11) 192.168.122.10 x (buth11) 192.168.122.10 x terraform: bash x (buth11) 192.168.122.10 x
Plan: 6 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ action_group_id      = (known after apply)
+ budget_name          = "SRE-Lab-Budget"
+ log_analytics_workspace_id = (known after apply)
+ log_analytics_workspace_key = (sensitive value)
+ resource_group_id    = (known after apply)
+ resource_group_name  = "SRE-Lab-RG"
+ subscription_id      = "b023e8b8-4446-4d4f-9fcf-d20a6c1fa093"

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
  terraform apply "tfplan"
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$ terraform apply tfplan
random_string.suffix: Creating...
random_string.suffix: Creation complete after 0s [id=5plwbb]
azurerm_resource_group.sre_lab: Creating...
azurerm_consumption_budget_subscription.sre_lab: Creating...
azurerm_consumption_budget_subscription.sre_lab: Still creating... [00m10s elapsed]
azurerm_consumption_budget_subscription.sre_lab: Creation complete after 19s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/providers/Microsoft.Consumption/budgets/SRE-Lab-Budget]

Error: A resource with the ID "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG" already exists -
to be managed via Terraform this resource needs to be imported into the State. Please see the resource documentation for "azurerm_resource_group" for more information.

with azurerm_resource_group.sre_lab,
on main.tf line 26, in resource "azurerm_resource_group" "sre_lab":
26: resource "azurerm_resource_group" "sre_lab" {

buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Zrzut Terraform Plan: 6 zasobów do utworzenia, 0 do zmiany, 0 do usunięcia. Widoczne szczegóły każdego zasobu wraz z tagami: environment=lab, managed_by=terraform, owner=bartosz.suszko.

```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × terraform: bash × (buth11) 192.168.122.10 ×

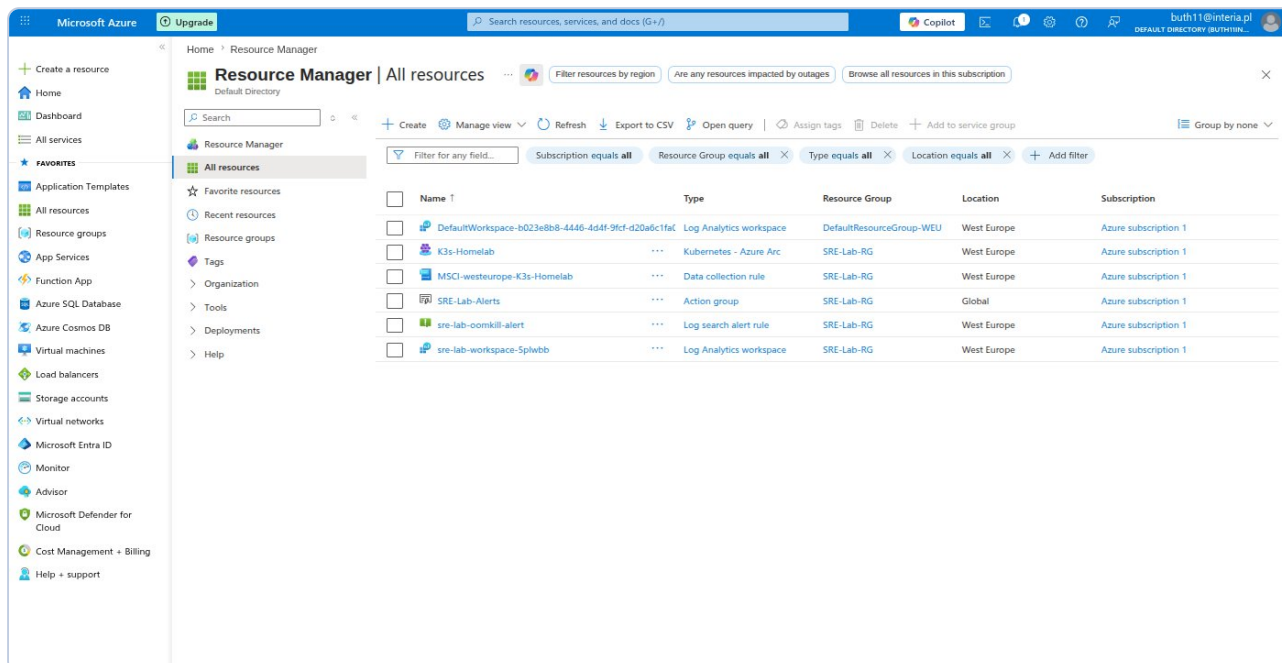
Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
terraform apply "tfplan"
azurerm_resource_group.sre_lab: Modifying... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurerm_resource_group.sre_lab: Modifications complete after 3s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurerm_monitor_action_group.sre_alerts: Creating...
azurerm_log_analytics_workspace.sre_lab: Creating...
azurerm_monitor_action_group.sre_alerts: Creation complete after 4s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m10s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m20s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m30s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m40s elapsed]
azurerm_log_analytics_workspace.sre_lab: Creation complete after 47s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb]
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creating...
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creation complete after 5s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/scheduledQueryRules/sre-lab-oomkill-alert]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

Outputs:
action_group_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts"
budget_name = "SRE-Lab-Budget"
log_analytics_workspace_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb"
log_analytics_workspace_key = <sensitive>
resource_group_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG"
resource_group_name = "SRE-Lab-RG"
subscription_id = "b023e8b8-4446-4d4f-9fcf-d20a6c1fa093"
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Zrzut Terraform Apply: Apply complete! Resources: 3 added, 1 changed, 0 destroyed. Widoczne outputy: action_group_id, budget_name, log_analytics_workspace_id.



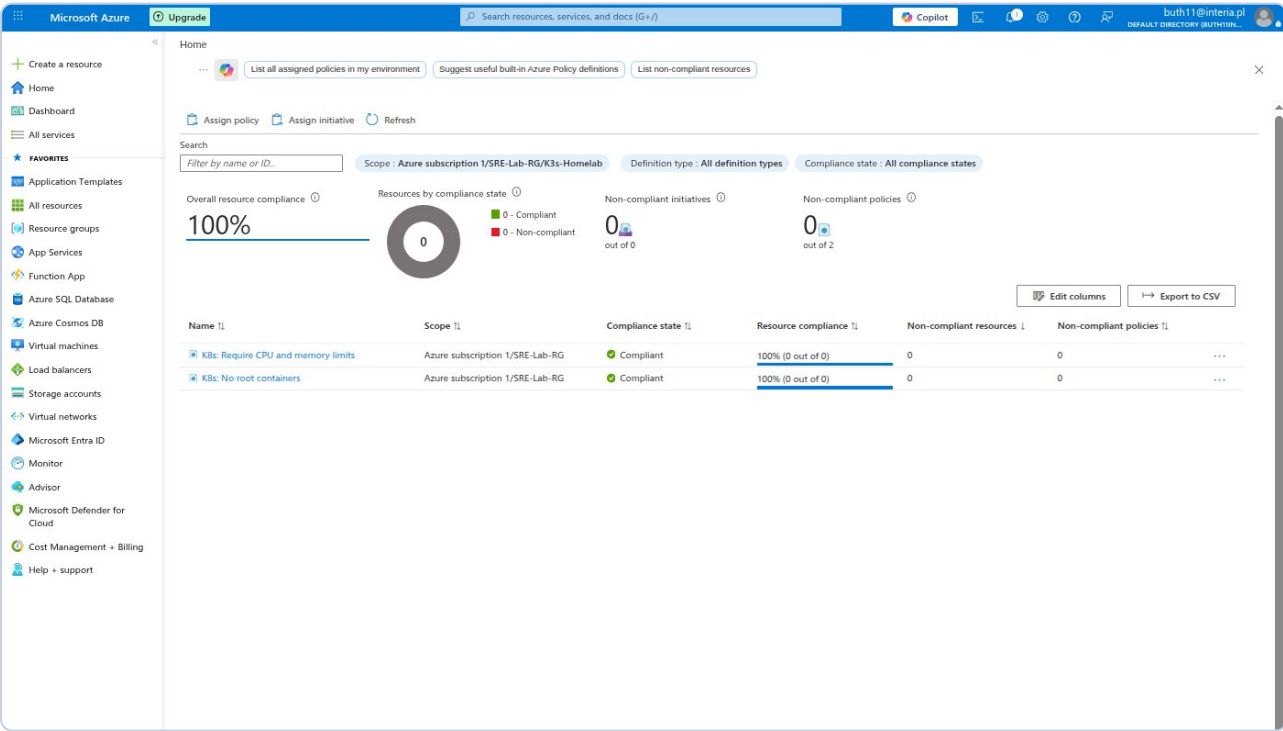
Zrzut Portal Azure — All Resources: widoczne zasoby utworzone przez Terraform w SRE-Lab-RG: SRE-Lab-Alerts (Action group), sre-lab-oomkill-alert (Log search alert rule), sre-lab-workspace-5plwbb (Log Analytics workspace).

Zmienne Terraform zawierają walidacje wejściowe (regex dla email, zakres wartości budżetu, lista dozwolonych regionów). Zasoby które już istniały w Azure (Resource Group) zostały adoptowane przez `terraform import` — to standardowa procedura przy wdrażaniu IaC na istniejącą infrastrukturę.

Rozdział 9b: Policy as Code — Azure Policy

Azure Policy umożliwia wymuszanie standardów bezpieczeństwa i zgodności na klastrach Kubernetes zarządzanych przez Arc. Dwie polityki zostały przypisane do Resource Group SRE-Lab-RG obejmując klaster K3s-Homelab.

Polityka	Zakres	Parametry	Tryb
K8s: No root containers	SRE-Lab-RG	—	DoNotEnforce (Audit)
K8s: Require CPU and memory limits	SRE-Lab-RG	CPU: 500m, RAM: 512Mi	DoNotEnforce (Audit)



Zrzut Azure Policy — Dashboard zgodności klastra K3s-Homelab: Overall compliance 100%, obie polityki widoczne (K8s: Require CPU and memory limits, K8s: No root containers), status "Not started" — agent Policy przeprowadza pierwszą ocenę klastra.

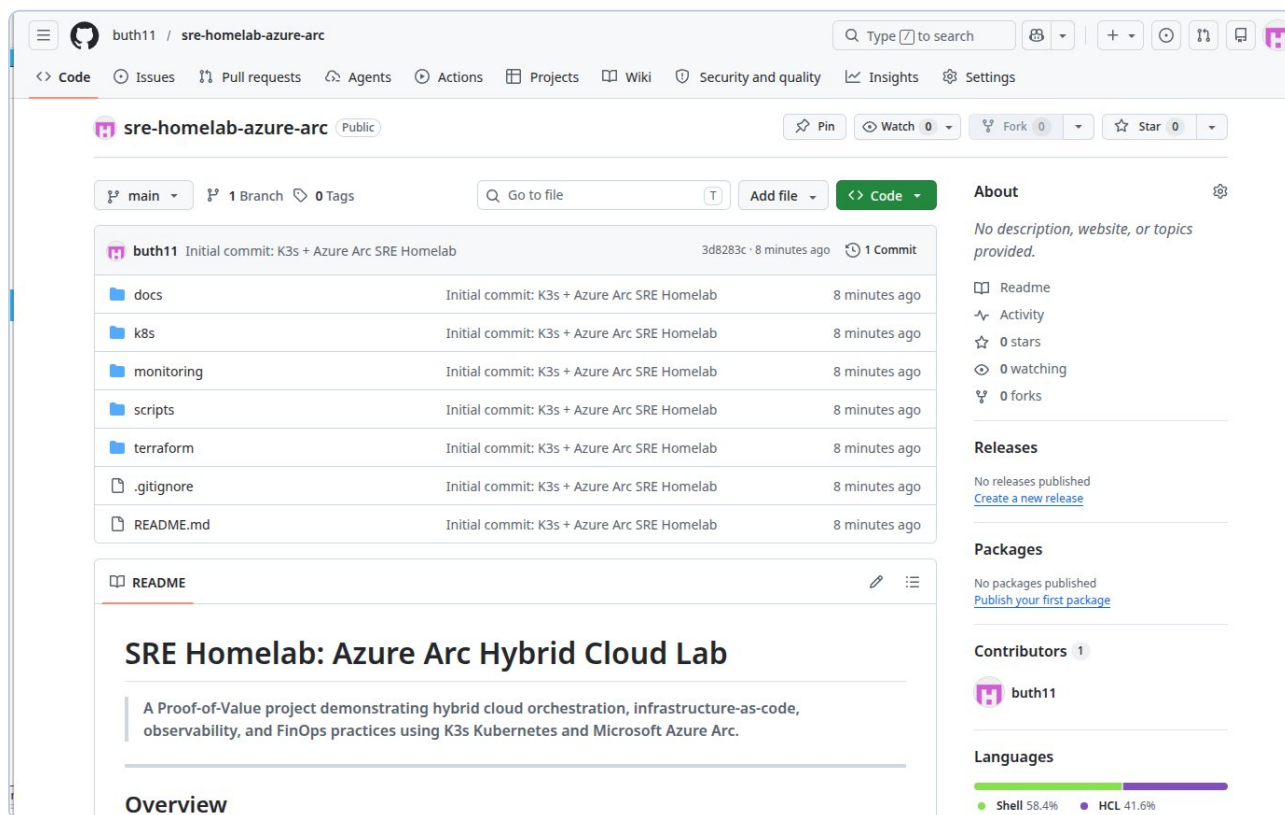
Tryb **DoNotEnforce** (Audit) pozwala na obserwację naruszeń bez blokowania workloadów — jest to zalecane podejście przy pierwszym wdrożeniu polityk na istniejące środowisko. Po analizie wyników tryb można zmienić na **Deny** dla nowych zasobów. Polityki zostały przypisane przez Azure CLI w jednej komendzie, co umożliwia ich zarządzanie jako kod (Policy as Code).

Cała infrastruktura Azure zdefiniowana w kodzie Terraform zapewnia powtarzalność wdrożeń, audit trail zmian przez Git i eliminuje błędy manualne wynikające z pracy w portalu.

- **azurerm_resource_group** — SRE-Lab-RG w West Europe z tagami
- **azurerm_log_analytics_workspace** — retencja 30 dni, SKU PerGB2018
- **azurerm_consumption_budget_subscription** — 5 USD/mies. z alertami 50% i 100%
- **azurerm_monitor_action_group** — email na adres SRE
- **azurerm_monitor_scheduled_query_rules_alert_v2** — alert OOMKill (KQL)

Zmienne Terraform zawierają walidacje (regex dla email, zakres wartości dla budżetu, lista dozwolonych regionów), co zapobiega błędnej konfiguracji już na etapie `terraform validate`.

Rozdział 10: GitHub — dokumentacja i automatyzacja



Zrzut 7: GitHub repo sre-homelab-azure-arc — publiczne, struktura: docs/, k8s/, monitoring/, scripts/, terraform/. Języki: Shell 58.4%, HCL 41.6%.

README.md napisany w języku angielskim zawiera: diagram architektury Mermaid, tabelę stosu technologicznego, rzeczywiste wyniki pomiarów oraz kompletne instrukcje reprodukcji środowiska krok po kroku.

Rozdział 11: Postmortem — POST-001 etcd Split-Brain

Pole	Wartość
ID incydentu	POST-001
Severity	P2 — Degraded cluster capacity
Czas trwania	Poniżej 7 minut (detekcja → rozwiązanie)
Impact	k3s-worker1 niedostępny, klaster na 2/3 pojemności
Root Cause	Literówka w nazwie hosta: ks3-worker1 zamiast k3s-worker1
Status	Rozwiązany — action items wdrożone

Literówka przy provisjoningu węzła (ks3 zamiast k3s) spowodowała rejestrację złej tożsamości w etcd. Próba zmiany nazwy przez `hostnamectl` wyzwoliła mechanizm ochronny etcd (Split-Brain prevention). Kluczowa lekcja: etcd traktuje tożsamość węzła jako niemutowalną — jedyne bezpieczne rozwiązanie to usunięcie wpisu przez API Kubernetes i ponowna rejestracja od zera. Skrypty instalacyjne zostały zaktualizowane o walidację regex nazwy hosta jako preflight check.

Rozdział 12: Wyniki i wnioski

Metryka	Wartość
Węzły klastra	3 / 3 Ready
Uruchomione pody	29 (Arc + AMA + K3s + App)
CPU utilisation	avg 2.20% / max 4.84%
RAM utilisation	avg 12.9% / max 14.48%
SLO Availability (24h)	100% — 184/184 próbek Ready
Arc agent uptime	12 agentów, 0 błędów krytycznych
Load balancer	Round-robin na 3 węzły potwierdzony
Koszty Azure	0.00 USD przez cały okres projektu
Czas resolucji POST-001	Poniżej 7 minut
Pliki IaC (Terraform)	5 zasobów z walidacją zmiennych

Projekt demonstruje pełny cykl życia infrastruktury SRE: projektowanie, wdrożenie, automatyzacja, observability i reagowanie na incydenty. Wszystkie komponenty stosu technologicznego wymaganego przez stanowisko Senior SRE zostały zainstalowane i zweryfikowane na rzeczywistym środowisku hybrydowym.